

Planificador de tareas para un sistema hidropónico automatizado programado en C++

Mauricio Yamil Tame Soria

18 de junio de 2018

Índice

1. Introducción	2
2. Configuración del Pcdduino	2
2.1. Paquetes	3
2.2. Configuración de red	4
3. Drivers	4
3.1. Protocolo para añadir drivers	5
3.2. Sensores de temperatura en líquidos	6
3.3. Sensor ph	7
3.4. Temperatura y Humedad ambiente	7
3.5. Humedad en tierra	7
4. Base de datos	8
4.1. Información del sistema	8
4.2. Monitoreo	8
4.3. Estadísticas	9
4.4. Conector c++ con mysql	9
5. Herramientas del sistema	9
6. Pruebas hardware	10
7. Planificador de tareas	10
7.1. Captura de datos	11
7.2. Rutinas de los sensores	11
7.3. Actuadores	12
7.4. Semáforos para los threads de muestreos	13
7.5. Recuperador de threads	14
8. Generador de estadísticas	14
9. Administrador de sistemas	15
9.1. Opción 2	15
9.2. Opción 3	16
9.3. Opción 4	16
9.4. Opción 5	16
10. Makefile	16
11. Interfaz web	17
11.1. Sistemas	17
11.2. Planificaciones y Riegos	18
11.3. Monitoreo	18
11.4. Estadísticas	18

1. Introducción

Se tiene un sistema hidropónico donde se busca automatizar los procesos que mantengan las condiciones óptimas para el cultivo. Los procesos consisten en regular, mediante actuadores, las variables del entorno: temperatura ambiente, temperatura en la solución de riego, humedad relativa en el ambiente, humedad relativa en la tierra y el ph de la solución de riego. Para mantener controladas las variables, es necesario el monitoreo de las mismas mediante muestreos. Dichas variables son de carácter cuantitativo, cuando se haga referencia a ellas, serán números reales (enteros con decimal). El monitoreo consiste en hacer muestreos de las variables periódicamente. Los valores monitoreados, igual que toda la información del sistema, son registrados en una base de datos. En los cultivos hidropónicos, la planta se nutre directamente del riego por lo que es importante mantener húmeda la raíz; se proponen dos formas de riego para cumplir esta condición. La automatización del sistema se lleva a cabo con el planificador de tareas, también nos referiremos a él como "scheduller". El planificador de tareas es un programa que mantiene los vegetales regados y procesa las variables del entorno para llevar a cabo los procesos que las mantengan en el rango ideal. Cada cultivo tiene un rango ideal de ph, temperatura ambiente, humedad ambiente y temperatura en la solución de riego; todos ellos se encuentran definidos en la base de datos. El sistema hidropónico está montado en un microprocesador "Pcduino"; los componentes (sensores y actuadores) del sistema están conectados a los pines de la tarjeta. El planificador de tareas, al igual que todos los programas, fue programado y compilado en el microprocesador; los códigos se encuentran en la carpeta principal del proyecto:

`/home/Hidroponia`

A lo largo del texto se tratarán los siguientes temas: configuración del microprocesador, métodos de muestreo, procesos para regular las variables del entorno, la base de datos del sistema, la estructura y funcionamiento del "scheduller", pruebas de hardware, los problemas del multithreading y sus soluciones, administración de los sistemas, estadísticas con las variables de entorno, una interfaz de usuario y las áreas de mejora del proyecto.

2. Configuración del Pcduino

La configuración del Pcduino consiste en la instalación de un sistema operativo, en este caso una distribución de Linux (lubuntu), y un conjunto de programas que nos permiten montar el servidor y administrar la base de datos. Para la instalación del sistema operativo lubuntu se deben obtener los archivos del kernel y el sistema operativo; se encuentran descargables en la página del distribuidor del Pcduino:

`http://www.linksprite.com`

Una vez descargados los archivos, se debe grabar el archivo imagen del kernel en una microSD con la ayuda del comando `dd` que provee Linux y guardar el archivo `ubuntu.iso` en una USB. Cuando esté la imagen del kernel grabada en la microSD, con el Pcduino apagado, se inserta la microSD y se conecta el

Pcduino a la corriente. Uno de los leds parpadea mientras se realiza la instalación del kernel; cuando deje de parpadear el led se debe retirar la microSD y presionar el botón "reset" del duino. Al reiniciar el duino, se observa una pantalla pidiendo insertar una USB con la imagen del sistema; se inserta la USB con el archivo **ubuntu.iso** y la instalación del sistema operativo comenzará a ejecutarse. Cuando termine el proceso se mostrará un mensaje en la pantalla, se retira la USB y se reinicia el duino para finalizar. Una vez instalado el sistema operativo, se necesita establecer una contraseña; para esto se abre la línea de comandos (Terminal) y se teclea:

```
sudo passwd
```

ya establecida la contraseña se reinicia el Pcduino. Establecer la contraseña es importante pues de lo contrario se presentan problemas al momento de instalar algunos paquetes. Para configurar la zona horaria, el tamaño de la pantalla y otras preferencias del sistema se ejecuta el comando:

```
sudo board-config.sh
```

2.1. Paquetes

Los programas necesarios para el funcionamiento del servidor son Apache, Mysql para la base de datos, php para la aplicación web (interfaz de usuario) y phpmyadmin para auxiliarnos con la base de datos. Todos los paquetes se instalaron desde la terminal usando el comando:

```
sudo apt-get install nombre_del_paquete
```

los nombres de los paquetes son:

- apache2
- php5
- libapache2-mod-php5
- mysql-server
- mysql-client
- phpmyadmin
- libmysqlcppconn-dev

Antes de instalar los paquetes, se debe ejecutar el comando **sudo apt-get update**. Una vez instalados los paquetes, es necesario reiniciar el duino. Al reiniciar, se nota que el servidor no funciona; el error se debe a que no se crea la carpeta **/var/log/apache2** al momento de iniciar el sistema operativo. Una forma sencilla de reparar el "bug", es editar el archivo **/etc/rc.local** agregando las siguientes líneas de código:

```
mkdir /var/log/apache2  
apachectl restart
```

La carpeta que contiene el phpmyadmin se encuentra en `/usr/share/phpmyadmin`; para acceder a ella del navegador, se debe crear un link simbólico dentro de la carpeta del servidor. Para ello, se ejecuta la siguiente línea de código:

```
sudo ln -s /usr/share/phpmyadmin /var/www
```

después de esto será posible acceder a la aplicación `phpmyadmin` desde el navegador con la ruta

`http://localhost/phpmyadmin`

Una vez instalados todos los paquetes, es necesario reiniciar el sistema. Tras seguir los pasos anteriores el Pcduno estará configurado para el sistema hidropónico.

2.2. Configuración de red

El servidor necesita una ip fija; para esto se debe abrir el menú de inicio, pasar a preferencias y luego pasar a las conexiones de red donde se edita la conexión que se utilizará. En este caso, se utiliza la conexión por cable ethernet. Para fijar la dirección ip se debe pasar a la pestaña `ipv4` (internet protocol version 4) y registrar los datos correspondientes. Actualmente el Pcduno tiene la siguiente configuración de red:

- ip: 10.10.254.78
- Netmask: 255.255.0.0
- Gateway: 10.10.255.254
- DNS server: 8.8.8.8
- Search domain: iies.unam.mx

Se debe reiniciar el Pcduno para que tomen efecto las configuraciones.

3. Drivers

Los drivers son los programas que permiten la comunicación del sistema con los sensores y actuadores. Los drivers son funciones definidas en archivos `cpp` y declaradas en archivos `hpp`. El formato que se escogió para declarar funciones de lectura fue el siguiente:

```
float driver_sensor(int pin);
```

donde el parámetro `pin` se refiere al puerto donde está conectado el sensor. Para el sensor de temperatura en agua, la función es diferente; el parámetro de entrada es el identificador del sensor y todos los sensores van conectados en paralelo al puerto GPIO 3. GPIO (General Purpose Input/Output, Entrada/Salida de propósito General) es un pin genérico en un chip, cuyo comportamiento (incluyendo si es un pin de entrada o salida) se puede controlar (programar) por el usuario en tiempo de ejecución. Para los actuadores, se utilizaron 3 funciones:

```
void actuar(int seg,int pin,string mensaje)
void activar(int pin,int pinData,int pinMode)
void parar(int pinData,int pinMode)
```

La función `actuar` recibe los siguientes parámetros: `seg` son los segundos que debe funcionar el actuador, `pin` es el puerto donde está conectado el actuador y `mensaje` es un texto que sirve para llevar un registro de lo que se ejecuta. La función `activar` sirve para encender el actuador por un tiempo indefinido y recibe los siguientes parámetros: `pin` el puerto donde está conectado el actuador, `pinData` es el identificador del archivo donde se almacena la información del puerto y `pinMode` es el identificador del archivo donde se define si el puerto es de entrada o salida. Por último, la función `parar` es la contraparte de `activar`, es decir, detiene un actuador que se encuentre activado. Para mayor detalle, todos los drivers se encuentran en la carpeta

`Hidroponia/Codes/Drivers`

excepto el driver de temperatura en líquidos, éste se encuentra en la dirección

`Support/c_environment/sample/temagua.c`

dentro de la carpeta del proyecto.

3.1. Protocolo para añadir drivers

El protocolo consiste en agregar dos archivos a la carpeta

`Hidroponia/Codes/Drivers`

un archivo `driver.hpp` que contenga la declaración de las funciones con el formato anterior y otro archivo `driver.cpp` donde estén definidas las funciones del driver. Una vez agregados los archivos, se compila el driver ejecutando el binario `comp` dentro de la carpeta `Hidroponia`. El binario `comp` se genera del código `compiler.cpp`, ubicado en la carpeta `Codes`. Para crear el binario, es necesario ejecutar dentro de la carpeta `Hidroponia` el siguiente comando:

```
make compiler
```

El binario `comp` compila todos los archivos `driver.cpp` que se encuentren en la carpeta `Drivers` y a partir de ellos genera los archivos objeto `driver.o`; estos son enviados a la carpeta `Hidroponia/Objects`. Una vez que están hechos los objetos, el programa `comp` genera un archivo de librerías llamado `libs.hpp` donde se escriben todos los nombres de los archivos `driver.hpp` que se encuentren en `Drivers`, es decir, se genera un archivo que contiene los headers de todos los drivers. El archivo `libs.hpp`, es el que se importa desde los otros códigos para poder usar las funciones de los drivers. Cuando termina de ejecutarse `comp`, se tienen los objetos que proveen las funciones de los drivers y un archivo `libs.hpp` para usarlo desde las otras partes del proyecto. El protocolo se pretende automatizar usando el administrador de sistemas; sin embargo la automatización continúa en desarrollo y más adelante se expondrá tanto el progreso alcanzado como los pasos para finalizarlo.

3.2. Sensores de temperatura en líquidos

Los sensores de temperatura en líquidos son controlados por un driver de código libre que se encuentra en Github; las instrucciones para su instalación están en la página del distribuidor del duino (linksprite). Hay que descargar una carpeta y ejecutar el comando **make** que está definido en el archivo **makefile** dentro de la carpeta que se descargó previamente. El nombre de la carpeta es **c_environment** y se colocó en la ruta

Hidroponia/Support/c_environment

En la Carpeta Support se colocan las herramientas que estén programadas en otro lenguaje; este driver está programado en *C*. Dentro de la carpeta

c_environment/sample

se encuentran algunos drivers para diferentes sensores arduino. Para este driver se utiliza el archivo llamado

DS18x20_Temperature.c

Dicho archivo genera un programa que captura lecturas de múltiples sensores conectados en un sólo puerto GPIO de manera continua y casi aleatoria. El programa regresa varios datos para cada lectura: el identificador del sensor, un valor en hexadecimal para verificar que la lectura sea correcta y la lectura del sensor en celcius. Este driver no permite ejecutar lecturas de un sensor determinado, por lo que fue necesario editar el código. Se creó una copia llamada

temagua.c

para realizar sobre ella la edición del código y conservar el original. El código se modificó para que se tomara sólo una lectura confiable checando el hexadecimal de verificación; dicha lectura se guarda en un archivo junto con el identificador del sensor que la realizó. Para crear los binarios de los drivers encontrados en la carpeta **c_environment/sample**, se debe ejecutar dentro de la carpeta **c_environment** el comando

make

para ejecutar el script en el archivo **Makefile** situado en la carpeta. El script compila las librerías **Arduino** necesarias para los drivers. Una vez que termina el proceso de compilación de las librerías, se debe pasar a la carpeta **c_environment/sample** para editar el archivo **Makefile** añadiendo la siguiente línea de código justo después de la última línea análoga:

OBJS += temagua

Esta línea de código representa el comando para que se compile el archivo **temagua.c** y se genere el ejecutable **temagua**. Una vez editado el archivo **Makefile**, dentro de **sample** se teclea el comando **make** para ejecutar el script y crear el binario el cual es enviado a la carpeta **c_environment/output/test**. Para usar el driver en el planificador de tareas, se creó una función en *C++* que obtiene la lectura de un determinado sensor. El pseudocódigo de la función es el siguiente:

```

Temperatura_en_liquido(sensor.id){
    ejecutar temagua;
    leer archivo;
    while(lectura.id!=sensor.id){
        ejecutar temagua;
        leer archivo;
    }
    return lectura;
}fin driver

```

Esta función representa el driver final, es decir, el que será utilizado por los demás programas. La función devuelve la lectura en tipo `float` y el parámetro `sensor.id` corresponde al identificador del sensor con el cual se efectuará la lectura. La declaración y definición de la función se hizo en la carpeta `Drivers` sobre los archivos `temagua.hpp` y `temagua.cpp` respectivamente.

3.3. Sensor ph

El sensor de ph es un sensor de conversión analógica-digital (adc en inglés). Para la calibración, se tomaron 3 valores conocidos de ph y se ejecutó la lectura del sensor sin el driver para cada uno de ellos, obteniendo así 3 puntos experimentales. Cada punto experimental representa un par ordenado de voltaje medido por el sensor y el ph de la muestra. Una vez obtenidos los 3 puntos experimentales, se realizó un ajuste lineal para obtener una recta que representa el ph en función del voltaje medido por el sensor adc. La función quedó de la siguiente manera:

$$ph(V) = V \frac{14 * 0,8}{4096} \quad (1)$$

donde V representa el voltaje registrado en el archivo correspondiente al puerto.

3.4. Temperatura y Humedad ambiente

La temperatura y la humedad ambiente las provee un sólo sensor, el DHT22 de arduino. Como el sensor es digital, no se utilizan los puertos adc sino los GPIO. Lo que hace el driver es fijar el pin en modo output para después capturar los bytes y procesarlos. El procesamiento de bytes devuelve lecturas de temperatura en celcius y humedad relativa en porcentaje (de 100). Aunque es un sólo driver, se crearon dos funciones: una para obtener la temperatura y otra para obtener la humedad. Las funciones se encuentran definidas y declaradas en los archivos `temhum.cpp` y `temhum.hpp`. La separación de las funciones tiene el fin de seguir el protocolo de funcionamiento de los drivers.

3.5. Humedad en tierra

El sensor de humedad en tierra es parecido al de ph, es del formato adc. El sensor consiste en 2 placas metálicas conectadas a una diferencia de potencial; cuando están sumergidas en agua, se registra un voltaje V_w al que le asociamos el 100% de humedad y luego, cuando las placas están al aire, se registra un

voltaje V_a al que le asociamos el 0 % de humedad; por lo tanto hay 2 puntos $(V_w, 100 \%)$ y $(V_a, 0 \%)$ con los cuales se hace un ajuste lineal (una recta que pase por los dos) y se obtiene:

$$H(x) = \frac{100}{V_w - V_a}(V - V_a) \quad (2)$$

donde H es la humedad en la tierra y V es el voltaje registrado por el sensor. Este driver era utilizado para el sensor FC-28 de arduino, ahora se utiliza un sensor similar creado por Diego Cabrer. Con el nuevo sensor, el driver responde correctamente al 0 % y al 100 %; para los valores intermedios arroja lecturas lógicas, sin embargo sería interesante calibrar el sensor con 9 muestras de tierra al 10 %, 20 %, 30 %, 40 %, 50 %, 60 %, 70 %, 80 %, 90 % de humedad y generar 11 puntos experimentales

$$(V_a, 0 \%), (V_{10\%}, 10 \%), \dots, (V_w, 100 \%) \quad (3)$$

para después hacer un ajuste (no necesariamente lineal) y obtener una mejor función $H(V)$.

4. Base de datos

La base de datos sirve para registrar toda la información de los sistemas: los actuadores y sensores, el tiempo y periodo de riego, la temperatura del agua, el rango de temperatura y humedad ambiente, entre otras cosas. La base también registra las lecturas de los sensores y las acciones que efectúan los actuadores. De las lecturas se procesan los promedios por hora y se guardan en una tabla de estadísticas.

4.1. Información del sistema

Para la información de los sistemas se tienen varias tablas. La tabla **sistema** guarda la información del sistema: nombre, cultivo, id, url, ubicación y si es sistema principal o subsistema. La tabla **sensor** tiene la información de los sensores: el tipo de sensor, el pin del microprocesador al que está conectado el sensor, el periodo de muestreo y el identificador del sistema al que pertenece. La tabla **actuador** tiene la información de los actuadores: el pin al que está conectado el actuador, el tipo de actuador y el identificador del sistema al que pertenece. La tabla **riego** contiene la información del riego: el tiempo de riego, el periodo de riego y la temperatura del agua con la que se riega. Existen 3 valores de tiempo, 3 valores para el periodo y 3 valores de temperatura del agua; cada uno está relacionado con la temperatura ambiente (la relación se explicará más adelante). La tabla **planificacion** contiene la información del cultivo: nombre del cultivo, rango ideal de temperatura y humedad ambiente, y rango de ph.

4.2. Monitoreo

Las tablas de monitoreos son

`monitoreo_actuador`, `monitoreo_sensor`

en la tabla `monitoreo_sensor` se registran fecha, hora, valor de la lectura e id del sensor; en la tabla `monitoreo_actuador` se registran fecha, hora, tipo de acción y valor de activación. El campo tipo acción de monitoreo actuador se refiere a encendido (1) o apagado (0).

4.3. Estadísticas

La tabla `estadistica_hora` se llena con los promedios por hora de los registros `monitoreo_sensor`. Los campos de esta tabla corresponden a la fecha, el tipo de sensor, el id sistema del cual se hacen los promedios y las horas del día. Dentro de los campos de las horas del día, se guardará el promedio de las lecturas que se hayan efectuado a esa hora, por ejemplo, un registro quedaría de la siguiente manera:

fecha	tipo	sistema	00	...	23
2018-05-15	ph	3	7.5	...	8.1

Este registro indica el promedio de ph del sistema con id= 3 a las 00 horas, 01 horas y así sucesivamente hasta mostrar el promedio de ph a las 23 horas. La tabla se actualiza mediante un trigger que se activa al introducirse un nuevo registro en la tabla `monitoreo_sensor`.

4.4. Conector c++ con mysql

Las consultas a la base de datos desde el planificador se hacen con `Mysql/C++ connector`, es un driver que provee Mysql. El driver se instala con el paquete `libmysqlcppconn-dev` en la parte de configuración. Para ejecutar consultas en paralelo, se creó una clase llamada `conector`, definida en el archivo `Codes/conector.cpp` y declarada en `Codes/Headers/conector.cpp`. Las consultas dentro del programa se ejecutan de la siguiente manera:

```
conector C;
C.conectar();
//para ejecutar sin resultado
C.stmt->execute("consulta");
//con resultado
C.res=C.stmt->executeQuery("consulta");
//...hacer cosas
C.borrar();
```

5. Herramientas del sistema

Las herramientas del sistema son un conjunto de estructuras y funciones usadas por el planificador de tareas y las pruebas del hardware; se encuentran declaradas y definidas en los archivos `Codes/Headers/tools.cpp` y `Codes/tools.cpp` respectivamente. Las estructuras más importantes son: `sensor`, `actuador` e `hilo`; fueron definidas de la siguiente manera:

```
struct sensor{
    int id,muestreo,pin,sys;
    float valor;
    string name;
};
```

```

struct actuador{
    int id,pin,seg,sys;
    string name;
};
struct hilo{
    string name;
    pid_t tid;
};

```

Las variables en la estructura **sensor** corresponden a la siguiente información:

- **id**: identificador del sensor, es un número entero.
- **muestreo**: minutos entre cada muestreo (periodo de muestreo).
- **pin**: pin al que está conectado el sensor.
- **sys**: identificador del sistema al que pertenece el sensor.
- **valor**: valor de la variable del entorno que representa el sensor.
- **name**: tipo de sensor, sirve para relacionar el sensor con el driver.

La estructura **actuador** es análoga a **sensor** para las variables de **id**, **pin**, **sys** y **name**. La variable diferente **seg**, representa los segundos de acción del actuador. La estructura **hilo** sirve para almacenar la información de un thread; la variable **name** guarda el nombre del thread y la variable **tid** (thread id) guarda el identificador del thread.

6. Pruebas hardware

Para verificar el funcionamiento de los sensores y actuadores, se creó el ejecutable **Hardware_test** definido en el archivo **Codes/pruebas.cpp**. El ejecutable carga los sensores y actuadores registrados en la base de datos dentro de las estructuras definidas anteriormente. Una vez creados los objetos con la información, se muestra un menú donde se decide si se desea probar los actuadores o los sensores. Posteriormente, se muestra un menú y se debe escoger si emitir una prueba general o escoger el sensor/actuador que se desee probar. La interfaz gráfica es muy rudimentaria por lo cual sería buena idea trabajar en este aspecto del proyecto. El ejecutable no tiene manera de hacer múltiples pruebas o de suspender la ejecución de una prueba; la solución sería ejecutar en threads las pruebas y regresar al menú para decidir si ejecutar otra prueba o terminar alguna.

7. Planificador de tareas

El planificador de tareas es un ejecutable llamado **Hidroponia** creado del código **scheduller.cpp** en la carpeta **Codes**. El ejecutable tiene un parámetro entero que representa el identificador del sistema que se va a activar. Por ejemplo, dentro de la carpeta **Hidroponia**, ejecutando el comando:

./Hidroponia 2

se inicia el planificador de tareas para el sistema con identificador 2 en la tabla **sistema** en la base de datos. Las tareas del sistema son las siguientes:

1. Capturar información de la base
2. Programar rutinas de los sensores
3. Programar el riego y los actuadores
4. Monitoreo de sensores y actuadores
5. Gestionar la caída de algún thread

7.1. Captura de datos

La captura de datos comienza por la información del sistema; partiendo del id se consulta lo que hay en la tabla **sistema** y se guarda en una estructura llamada **sistema**. Después se consulta su planificador y el riego para almacenar los datos en una clase llamada **planificador**, declarada en **tools.hpp** y definida en **tools.cpp**. Posteriormente, se cuenta la cantidad de sensores y actuadores para crear dos arreglos de estructuras **sensor** y **actuador** de tamaños respectivos; creados los arreglos, se inicializan con la información de cada sensor del sistema. Esta fase del programa consiste en guardar la información de la base de datos en las estructuras definidas en las herramientas del sistema (**tools.cpp**).

7.2. Rutinas de los sensores

Lo siguiente es ejecutar las rutinas de muestreo; para cada sensor se lanza un thread que realiza dicha rutina. El pseudocódigo del thread de muestreo es el siguiente:

```
Muestreo(sensor){
    loop{
        esperar el pin;
        ocupar pin;
        inicializar sensor.valor=-1;
        intentos=0;
        while(sensor.valor===-1){
            indexar el sensor con su driver
            if(sensor.name=tipo1){
                sensor.valor=driver_tipo1(sensor.pin);
            }
            else if(sensor.name=tipo2){
                sensor.valor=driver_tipo2(sensor.pin);
            }
            ...
            ...
        }
        else{
            no hay driver;
            exit;
        }
    }
}
```

```

    }
    if(sensor.valor== -1&intentos++<2){
        reportar error;
    }
}fin while
registrar lectura
en la base de datos;
liberar pin;
sleep(sensor.muestreo);
}fin loop
}fin muestreo

```

La edición de este thread es la última parte de la automatización del protocolo para añadir drivers; se debe colocar la función en un archivo separado para editarlo, actualmente se encuentra en `Codes/scheduller.cpp` junto con todo el planificador. Para continuar con el protocolo, se tendrían que añadir las siguientes líneas de código después del último `else if` dentro del thread:

```

else if(sensor.name=="tipo_nuevo"){
    sensor.valor=nuevo_driver(sensor.pin);
}

```

después compilar la función en un objeto y actualizar las librerías en el archivo `libs.hpp`. Finalmente, actualizar los ejecutables `Hardare_test` e `Hidroponia` para tener las pruebas del sistema y el planificador de tareas actualizados con el nuevo driver.

7.3. Actuadores

Los diferentes tipos de actuadores que soporta el sistema son:

1. bomba de agua
2. calefactor del agua
3. ventilador
4. reguladores de ph

y para cada uno está definido un thread que se lanza sólo si se cubren determinados requerimientos. Para el riego existen 2 threads: uno se ejecuta periódicamente de acuerdo a la información del riego en la base de datos y el otro se activa dependiendo la humedad de la tierra (si baja a determinado nivel se activa el riego hasta que la humedad vuelva a subir). El primer thread de riego sólo requiere una bomba de agua mientras que el otro requiere una bomba de agua y un sensor de humedad en tierra para poder ejecutarse. El thread del calefactor de agua requiere un actuador de tipo calefactor y un sensor de temperatura en líquidos; el principio es el mismo que el segundo riego, si el agua baja a cierta temperatura se activa el calefactor hasta que se consiga una temperatura en el rango asignado por el planificador. El ventilador es análogo, se activa a determinada temperatura ambiente y se desactiva cuando la temperatura ambiente llega al rango ideal. Los reguladores de ph no se instalaron pero el thread debe ser creado de la misma manera que los otros actuadores; para subir el ph, el pseudocódigo sería:

```

activar(regulador_phmas.pin);
while(sensorph.valor<phminimo){
    sleep(t);
    sensorph.valor=lectura de ph;
}
desactivar(regulador_phmas.pin3);

```

Para disminuir el ph, el pseudocódigo sería igual únicamente se modificaría la condición del while: `while(sensorph.valor>phmax)`. La letra `t` representa un pequeño periodo de muestreo para saber cuándo desactivar el actuador correspondiente.

7.4. Semáforos para los threads de muestreos

Dos de los problemas relacionados con el multithreading fueron deadlocks al momento de hacer consultas a la base y la concurrencia de threads que intentaban acceder al mismo espacio de memoria en un determinado instante; dichos problemas causaban el fallo del sistema operativo. El primer problema se resolvió con un thread (semáforo) encargado de ejecutar las consultas de monitoreo en orden. El semáforo de los monitoreos consiste en una fila donde se insertan las consultas en forma de strings y se van ejecutando una por una. Para el segundo problema, el semáforo consiste en un arreglo de valores booleanos y 3 pasos: esperar, usar y liberar. Cada elemento del arreglo corresponde a un pin del duino; si el elemento es verdadero quiere decir que el pin correspondiente al index del elemento está ocupado. Cuando un thread quiere usar el pin, debe consultar el arreglo para ver si está ocupado, de ser así debe esperar. La forma de usar un pin es la siguiente:

```

while(Pinbussy[4]){ //esperar }
//establecer el pin ocupado
Pinbussy[4]=true;

...usar el pin

//liberar el pin
Pinbussy[4]=false;

```

En este ejemplo, el thread espera en el loop `while` hasta que se libera el pin 4; saliendo del loop, se fija el valor en `true` para establecerlo como ocupado; posteriormente, se utiliza el pin y se libera. Los semáforos se encuentran definidos en un thread llamado `monitoreo` y en un conjunto de funciones llamadas `usar(pin)`, `liberar(pin)` y `esperar(pin)`. El último semáforo creado fue el administrador de sensores de temperatura en líquidos. El driver del sensor de temperatura en líquidos utiliza un archivo para su funcionamiento; cuando dos threads intentaban acceder al archivo al mismo tiempo, el sistema operativo colapsaba. El semáforo es un programa que consulta los sensores de temperatura en líquidos dados de alta para después efectuar lecturas con cada uno y posteriormente registrarlas en la base de datos. Las muestras y los registros se efectúan de manera ordenada, un sensor a la vez, para evitar la convergencia de threads en el archivo de lecturas.

7.5. Recuperador de threads

Otro problema que surgió con el multithreading fue la muerte del thread de monitoreo. A pesar del thread caído, el programa continuó en ejecución; no se registraban los muestreos a la base de datos monitoreo. En aquel momento no hubo manera de rastrear el error que ocasionó el fallo del thread. Para poder rastrear los errores y las caídas del thread fue necesario crear archivos log; uno para el registro de los procesos normales y otro para los errores. Los archivos log se crearon redirigiendo las variables `stdout` y `stderr` hacia los archivos `Hidroponia/Logs/logfile` y `Hidroponia/Logs/errfile` respectivamente. Cada planificador de tareas tiene su `logfile` y `errfile`, se encuentran diferenciados por el identificador del sistema al final del nombre del archivo. Para manejar la caída de threads se creó un reparador de threads dentro del loop principal del administrador de tareas. El pseudocódigo es el siguiente:

```
main(int id_planificador){
...cargar sensores y actuadores
n=no. de threads;
(usamos la estructura hilo)
arreglo de hilos[n];
...Ejecutar threads y guardar info
...en estructura hilo[i]
loop{
    for(i=0;i<n;i++){
        if(!running(hilos[i].tid){
            relanzar el thread;
            reportar en logfile;
        }end if
    }end for
    esperar media hora;
}end loop
}end main
```

En la rutina se verifica periódicamente si cae algún thread y, en caso de caerse, relanzarlo y reportar. La función `running(hilo.tid)` verifica si está corriendo un thread mediante su tid; la función se apoya de una carpeta que contiene archivos con información de status para cada thread que sea ejecutado; la carpeta es `Hidroponia/Threads`. El reparador de threads es el loop principal del planificador de tareas, mantiene el programa en ejecución; si llegara a caer el reparador de threads, caerían todos los threads con él. Hace falta manejar la caída del thread principal, debe ser programado en el sistema operativo; algún script que esté ejecutando periódicamente la opción 2 del administrador de sistemas.

8. Generador de estadísticas

El generador de estadísticas es un ejecutable llamado `stats` compilado del código `stats.cpp`. Lo que hace es generar periódicamente los promedios de las lecturas y registrarlos en la tabla `estadistica_hora`. El pseudocódigo del generador de estadísticas es el siguiente:

```

loop{
  consultar los sistemas activos;
  para cada sistema{
    consultar sensores registrados;
    para cada sensor{
      if(registros_sensor==0){
        crear registro;
      }
      avg=promedio de lecturas
        en la hora actual;
    }
  }
  update
  estadistica_hora.hora_actual=avg;
  }fin sensores
  }fin sistemas
  esperar 20 minutos;
}fin loop;

```

El programa fue reemplazado por el trigger que promedia en la hora con cada nuevo registro en la tabla `monitoreo_sensor`.

9. Administrador de sistemas

El administrador de sistemas es un ejecutable llamado `admin` definido en el código `manager.cpp` en la carpeta `Codes`. El administrador tiene 4 opciones, 3 de ellas funcionales y 1 en desarrollo. Las opciones se ejecutan sobre la carpeta principal `Hidroponia` de la siguiente manera:

```
./admin opc
```

donde `opc` es un entero que representa la opción que se desea ejecutar. Las opciones son: "2" para actualizar los sistemas con la base de datos, "3" para actualizar los drivers, "4" matar todo y "5" ejecutar las pruebas del hardware.

9.1. Opción 2

La opción 2 actualiza los "schedulers" en ejecución con la base de datos. El pseudocódigo es el siguiente:

```

load_systems(){
  get sistemas where estado=1;
  for each sistema{
    if(!running(sistema)){
      ./Hidroponia sistema.id;
      get subsistemas where estado=1;
      for each subsistema{
        if(!running(subsistema)){
          ./Hidroponia subsistema.id;
        }end if
      }end subsistemas
    }end if
  }end if
}

```



```

    }end sistemas
get sistemas with estado=0;
for each sistema{
    if(running(sistema)){
        kill(sistema);
        get subsistemas;
        for each subsistema{
            kill(subsistema);
        }end subsistemas
    }end if
}end sistemas
if(!running(stats)){ ./stats }
if(!running(wateradmin)){ ./wateradmin }
}end load_systems

```

Por lo tanto, si se quisiera apagar o encender un determinado sistema sería recomendable editar el campo **estado** en la base de datos y ejecutar la opción 2 del administrador de sistemas.

9.2. Opción 3

Esta opción sigue en desarrollo y se ha comentado a lo largo del documento. Sirve para automatizar el protocolo para añadir un driver al sistema. Lo que se tiene hasta ahora es la creación de los objetos con los archivos de la carpeta Drivers y la creación del archivo **libs.hpp** que contiene todos los headers; se logra con la ejecución del binario **comp** sobre la carpeta **Hidroponia**. Falta editar el código del thread de muestreo y actualizar los binarios **Hidroponia** y **Hardware_test**.

9.3. Opción 4

Esta opción consulta todos los sistemas registrados en la base de datos y apaga cada uno de ellos. También termina los programas **stats** y **wateradmin**. Se utiliza para reiniciar los sistemas, primero ejecutar opción 4 y luego opción 2.

9.4. Opción 5

Es la opción para ejecutar las pruebas del hardware. Esta opción primero termina todos los sistemas con la opción 4 y después ejecuta el binario **./Hardware_test** para correr las pruebas del hardware.

10. Makefile

El archivo **makefile**, ubicado en la carpeta principal, contiene las definiciones de los comandos de compilación para las partes del proyecto. Al ejecutar el comando **make**, se compilan todos los módulos del sistema hidropónico. Para compilar los módulos por separado, se ejecuta el comando **make modulo** donde los diferentes módulos son:

- **compiler:** compila el código `compiler.cpp` para hacer el ejecutable `comp`.
- **drivers:** ejecuta `./comp` para crear los objetos.
- **conector:** compila el archivo `conector.cpp` para la clase conector.
- **tools:** compila `tools.cpp`.
- **pruebas:** compila `pruebas.cpp` para crear `Hardware_test`.
- **hidroponia:** compila `scheduller.cpp` para crear el ejecutable `Hidroponia`.
- **manager:** compila `manager.cpp` para crear `admin`.
- **statistics:** compila `stats.cpp` para crear `stats`.
- **water:** compila `wateradmin.cpp` para crear `wateradmin`.
- **clean:** borra los ejecutables.

Por ejemplo, para actualizar el planificador de tareas, después de una modificación al código, se debe ejecutar `make hidroponia` sobre la carpeta principal `Hidroponia`.

11. Interfaz web

Se creó una pequeña interfaz gráfica provicional para agregar/editar sistemas, planificadores y riegos; también permite monitorear las lecturas de los sensores y las acciones de los actuadores así como las estadísticas generadas en la tabla `estadistica_hora`. Los códigos de la interfaz se encuentran en la carpeta

`/var/www/Proyecto/`

Para acceder a la interfaz desde el navegador, se debe ingresar en la barra de dirección lo siguiente:

`10.10.254.78/Proyecto`

Es importante resaltar que sería la ip que tuviera asignada el servidor.

11.1. Sistemas

En esta parte de la interfaz se registra/edita un sistema o subsistema. Al pasar a la página `sistema.php` se muestra la opción de editar un sistema o crear uno nuevo. Para la opción de editar se despliegan todos los sistemas registrados; si se desea editar un subsistema hay que escoger el sistema padre y después el subsistema. En la parte de edición del sistema o subsistema se encuentran las siguientes opciones:

- Añadir, quitar o modificar sensores y actuadores
- Editar la información del sistema
- Cambiar de planificador o riego
- Encender el sistema
- Borrar el sistema

11.2. Planificaciones y Riegos

Análogo a sistemas, se muestran las opciones de crear o de editar planificaciones y riegos. Los códigos de las páginas son `planificador.php` y `riego.php` respectivamente.

11.3. Monitoreo

Se encuentra en el archivo `monitoreo.php`; lo que hace el script es filtrar las tablas `monitoreo_sensor` y `monitoreo_actuador` preguntándole al usuario el sistema que desea monitorear, si desea monitorear sensores o actuadores, qué tipo de sensor o actuador y el intervalo que desea monitorear. Una vez que se fijan esas condiciones, se genera una consulta a la base de datos y se crean páginas de 30 registros para visualizar las tablas de monitoreo filtradas. Para mayor detalle, consultar el código `monitoreo.php` en la carpeta de la interfaz.

11.4. Estadísticas

En esta parte de la interfaz se pueden visualizar los promedios de las variables de entorno respecto cada hora, día, semana o mes. Para la visualización de promedios por hora se hace una consulta a la base de datos de la tabla `estadistica_hora` donde se guardan y se generan los promedios de las lecturas que están en `monitoreo_sensor` de manera ordenada en registros diarios. Para la visualización en días se consulta a la tabla `monitoreo_sensor` el promedio de los valores de cierto sensor a lo largo del día, análogamente para semana y mes. Para mayor detalle de las consultas, revisar el código `estadisticas.php` en la carpeta de la interfaz.